

第1章 HDFS | 开卷速查版

本章一句话：**HDFS** 是一个面向大文件、顺序读写、高吞吐、可容错的分布式文件系统。它把一个大文件切成多个 **Block**，分散存到多台 **DataNode** 上，由 **NameNode** 统一管理元数据。☺

常考题型： 简答题为主，常问“为什么需要HDFS”“为什么Block这么大”“NameNode/DataNode/Client分别做什么”“写流程”“副本放置”“为什么不适合小文件”。

翻页关键词： 单机文件系统 → HDFS特点/局限 → 架构 → Block → DataNode存储 → NameNode元数据 → BlockMap → 写流程 → 副本策略 → 容错 → 小文件问题。

1. 单机文件系统：以 Linux ext2 为例

单机文件系统就是数据都存在**一台机器**的磁盘上。PPT用 Linux ext2 举例，它主要有三个部分：**super block**、**inode**、**data block**。

super block，超级块 / 管理块：记录整个文件系统的管理信息，比如有多少数据块、多少 inode、哪些已经用掉。可以理解成“磁盘账本”。

inode，索引节点 / 元数据：每个文件对应一个 inode。inode 不直接存文件内容，而是记录文件属性和文件内容放在哪些数据块里。比如文件大小、权限、时间、指向哪些 data block。

data block，数据块：真正存文件内容。Linux ext2 里常见数据块大小是 **4KB**。磁盘读数据时以 block 为单位读，每读一次就有一次磁盘 I/O 开销。PPT明确把 ext2 的 super block、inode、data block 作为单机文件系统的基本结构，其中 data block 是 4KB。

单机文件系统的问题：数据规模变大以后，单机的**容量**会成为瓶颈，单机的 **I/O** 会成为瓶颈，机器一旦故障，数据可能不可访问。PPT里总结为：容量瓶颈、I/O瓶颈、机器故障导致数据不可访问。

考试写法：单机文件系统的数据和计算都集中在一台服务器上，适合小规模数据管理；当数据规模扩大时，会遇到存储容量不足、I/O性能不足和单点故障风险高的问题，因此需要分布式文件系统。

2. 为什么单机 Block 是 4KB，HDFS Block 是 64MB？

单机文件系统的 **block 小，一般 4KB**。因为单机文件系统要兼顾大量小文件和随机读写。如果 block 太大，小文件也要占一个大块，会浪费空间。

HDFS 的 block 大，PPT里是 64MB，后续版本常见 128MB。因为 HDFS 面向的是大文件和顺序读写。大 block 可以减少 block 数量，也就减少 **NameNode** 需要维护的元数据数量；同时可以减少磁盘寻道，让一次读写尽量连续进行，提高吞吐量。PPT中直接说明：传统文件系统 block 为 4KB，HDFS block 为 64MB，原因是减少磁盘寻道、提高顺序读写吞吐量、适合大规模数据处理。

核心理解：4KB 是为了“细粒度管理、少浪费”；64MB 是为了“大文件顺序处理、高吞吐、少元数据”。

考试写法：HDFS 的 block 远大于单机文件系统，是因为 HDFS 主要处理 GB/TB/PB 级大文件，并且以顺序读写和批处理为主。较大的 block 可以减少磁盘寻道次数，减少 NameNode 维护的 block 元数据数量，提高顺序读写吞吐量。但这也使 HDFS 不适合大量小文件。

3. HDFS 的基本特点

HDFS = Hadoop Distributed File System，即 Hadoop 分布式文件系统。它受 Google GFS 论文启发，目标是构建开源的大规模分布式文件系统，主要用于大规模数据存储和 MapReduce 大数据计算平台。

高容错性：数据自动保存多个副本。某个副本丢失后，系统可以自动恢复。

适合批处理：HDFS 强调“**移动计算而非数据**”。意思是不要把大数据搬来搬去，而是尽量把计算任务调度到数据所在机器上执行。

适合大数据处理：面向 GB、TB、PB 级数据，百万规模以上文件数量，甚至 10K+ 节点规模。

使用普通服务器：不依赖昂贵高可靠硬件，而是用普通机器 + 多副本 + 容错恢复机制保证可靠性。PPT 中把高容错、适合批处理、适合大数据、使用普通服务器列为 HDFS 基本特点。

考试写法：HDFS 具有高容错性、适合批处理、适合大数据处理、可运行在普通服务器上的特点。它通过多副本机制提高可靠性，通过暴露数据位置支持移动计算而非移动数据，因此适合大规模离线分析任务。

4. HDFS 的局限

不适合低延迟访问：HDFS 追求的是高吞吐，不追求毫秒级响应。它适合一次读写大量数据，不适合频繁的小请求。

不适合小文件存取：每个文件、每个 block 的元数据都要由 **NameNode** 维护。小文件数量太多，会占用 NameNode 大量内存；而且小文件读取时，寻道和调度开销可能超过真正读数据的时间。

不适合并发写入和随机修改：HDFS 不支持像普通数据库那样随机改文件内容，主要支持写一次、多次读，或者追加写入 append。PPT 中明确列出：不适合低延迟数据访问、不适合小文件存取、不适合并发写入和文件随机修改，只支持 append。

考试写法：HDFS 的局限来自它的设计目标。它为大文件顺序读写和高吞吐批处理优化，因此不适合毫秒级低延迟访问；大量小文件会占用 NameNode 内存并带来过高寻道开销；同时 HDFS 不支持文件随机修改，只适合追加写入和一次写多次读。

5. HDFS 的设计思想

HDFS 的核心思想可以压成四句话：

大文件切块：一个大文件被切成多个 **Block**。

分布式存储：不同 Block 分散存放在多台 **DataNode** 上。

多副本机制：每个 Block 保存多个副本，默认通常是 3 份，用来保证可靠性。

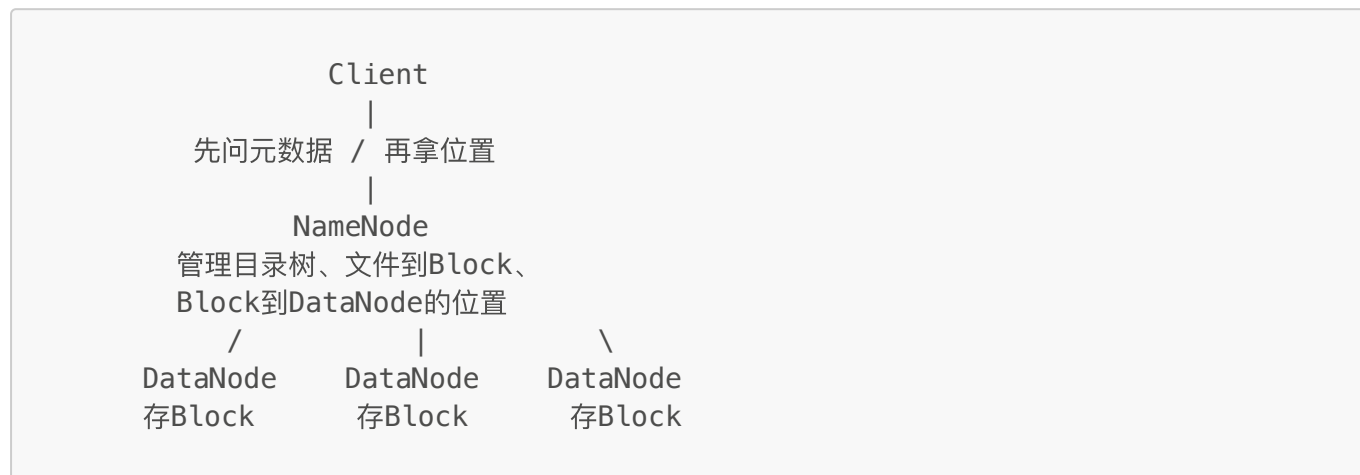
移动计算而非数据：数据很大，网络搬运很贵，所以尽量让计算任务去数据所在节点执行。PPT 中用磁盘吞吐和网卡吞吐对比说明移动计算的理由：机械硬盘吞吐可达 60-90MB/s，多块硬盘 300-500MB/s，而 100Mbps 网卡吞吐只有 12.5MB/s。

考试写法：HDFS 采用大文件切块、分布式存储、多副本容错和移动计算而非数据的设计思想。文件被切成多个 block 分散存储在不同 DataNode 上，NameNode 统一管理元数据，多个副本用于容错，计算框架尽量在数据所在节点运行任务以减少网络传输。

6. HDFS 架构：图怎么画、每部分做什么

HDFS 是典型的 **master-slave 主从架构**。系统主要有三类角色：**NameNode**、**DataNode**、**Client**。PPT 中明确写出：NameNode 管理文件系统元数据，DataNode 存储实际数据块，Client 连接 HDFS。

图可以画成这样：



NameNode：管理者。 它不存真实文件内容，而是管理元数据。包括目录树、文件属性、文件由哪些 Block 组成、每个 Block 存在哪些 DataNode 上。

DataNode：数据存储者。 它存真实的 Block 文件，并负责读写请求。DataNode 会定期向 NameNode 汇报自己活着，也会汇报自己保存了哪些 Block。

Client：用户入口。 Client 读写文件时，先向 NameNode 查询元数据和 block 位置，再直接和 DataNode 传输数据。也就是说，大数据流一般不经过 NameNode，否则 NameNode 会成为瓶颈。

考试写法： HDFS 采用主从架构，由 NameNode、DataNode 和 Client 组成。NameNode 管理文件系统命名空间和 block 位置信息；DataNode 存储实际 block 并处理读写；Client 先向 NameNode 请求元数据，再直接访问对应 DataNode 完成数据读写。

7. NameNode、DataNode、Client 分别是什么

NameNode = 元数据中心。 它知道“文件系统长什么样”。比如 `/user/a.txt` 这个文件是否存在、权限是什么、由哪些 block 构成、这些 block 分别在哪些 DataNode 上。

DataNode = block 仓库。 它只负责存 block 文件。它本身不知道这个 block 属于哪个完整 HDFS 文件。这个关系只有 NameNode 知道。

Client = 操作者。 Client 负责和 HDFS 交互。读文件时先问 NameNode：“这个文件的 block 在哪里？”然后去找 DataNode 读数据。写文件时先问 NameNode：“我能写吗？写到哪些 DataNode？”然后建立数据写入管道。

一句话记忆：**NameNode 管目录和位置，DataNode 管真实数据，Client 负责发起读写。**

8. 数据块 Block：是什么，为什么这么大

Block 是 HDFS 存储文件的基本单位。一个大文件会被切成多个 Block。比如一个 200MB 文件，如果 HDFS block 是 64MB，大概会切成 4 个 block，最后一个 block 不满也可以。

为什么这么大：

第一，减少磁盘寻道。HDFS 主要顺序读写，block 大可以让磁盘连续读一大段数据。

第二，提高吞吐量。一次读更多数据，适合批处理。

第三，减少 NameNode 元数据压力。如果 block 太小，同样一个大文件会产生更多 block，NameNode 要维护更多映射关系。

注意： HDFS 中小于 64MB 的文件不会真的占满 64MB 磁盘空间，但它仍然会占用 NameNode 中一份文件/block 元数据。因此小文件问题的核心不是磁盘浪费，而是 **NameNode 元数据压力** 和访问开销。

9. DataNode 上的数据怎么存

每个 **DataNode** 本质上是一台 Linux 服务器，上面运行 HDFS 进程，同时使用本地 Linux 文件系统存放 block。PPT 中说明：数据以 Block 文件形式存储在 Linux 本地文件系统中，每个 Block 对应一个真实数据文件和一个校验信息文件，例如 `blk_1073741825` 和 `blk_1073741825_xxx.meta`。

真实数据文件：如 `blk_xxx`，存 block 的真实内容。

校验文件：如 `blk_xxx.meta`，存 checksum，用来检查数据是否损坏。

关键点： DataNode 上看到的是很多普通 Linux 本地文件，它不知道这些 block 拼起来属于哪个 HDFS 文件。完整关系由 NameNode 维护。PPT 也强调，需要区分 HDFS 文件和 DataNode 上的本地文件，DataNode 不知道存储在它上面的 block 属于哪个 HDFS 文件。

考试写法： DataNode 使用本地 Linux 文件系统保存 HDFS block。每个 block 通常对应一个真实数据文件和一个 checksum 校验文件。DataNode 只保存 block 本身，并不知道 block 属于哪个完整 HDFS 文件，文件到 block 的映射关系由 NameNode 维护。

10. 元数据：NameNode 到底存什么

元数据就是“描述数据的数据”。HDFS 的真实文件内容在 DataNode 上，NameNode 存的是管理信息。

NameNode 元数据主要包括：

文件系统命名空间：目录树结构，比如有哪些目录、有哪些文件。

文件属性信息：文件名、权限、所有者、大小、修改时间等。

文件到 Block 的映射：比如 `a.txt` 被切成 `block1`、`block2`、`block3`。

Block 到 DataNode 的映射：比如 `block1` 的三个副本分别在 DN1、DN3、DN5。

NameNode 元数据可以理解为两大类：

1. 文件系统树：
 `/user/a.txt -> block1, block2, block3`

```
2. Block位置:  
block1 -> DN1, DN3, DN5  
block2 -> DN2, DN4, DN6
```

11. NameNode 元数据的存储结构

NameNode 的元数据不是只放在内存，也不是只放在磁盘，而是“内存 + 磁盘持久化”。

内存中：保存正在运行时需要快速查询的数据结构，比如目录树、INode、BlockMap。内存查询快，适合频繁处理 Client 请求。

磁盘中：保存 **FsImage** 和 **EditLog**。

FsImage：文件系统元数据的某个时刻快照。可以理解成“某一刻完整目录树和文件信息的存档”。

EditLog：修改日志。记录从 FsImage 之后发生的创建文件、删除文件、改名等操作。可以理解成“操作流水账”。

NameNode 重启时，会先加载 FsImage，再重放 EditLog，恢复出最新的文件系统元数据。

考试写法：NameNode 的元数据采用内存和磁盘结合的方式。运行时元数据放在内存中以便快速访问，磁盘上通过 FsImage 保存元数据快照，通过 EditLog 记录后续修改操作。NameNode 重启时加载 FsImage 并回放 EditLog 恢复命名空间。

12. BlockMap 是什么？为什么不持久化？

BlockMap 是 NameNode 内存中的一张映射表，用来记录：

```
BlockID -> 这个Block的副本在哪些DataNode上
```

例如：

```
block_001 -> DN1, DN4, DN8  
block_002 -> DN2, DN5, DN7
```

为什么 BlockMap 不持久化？

因为 BlockMap 记录的是 block 当前存在哪些 DataNode 上，而这个信息变化很频繁。DataNode 可能上线、下线、故障、重启，block 也可能复制、删除、迁移。如果每次变化都持久化，会产生大量磁盘 I/O，而且磁盘上的位置信息很容易过期。

HDFS 的做法是：NameNode 重启后，先通过 FsImage/EditLog 恢复“文件由哪些 block 组成”，再由各个 DataNode 主动发送 **BlockReport** 汇报“我这里有哪块 block”，NameNode 根据汇报重新构建 BlockMap。

考试写法：BlockMap 不持久化，因为 block 到 DataNode 的位置关系是动态变化的，DataNode 上下线和 block 复制迁移都会使其频繁变化。持久化 BlockMap 会带来额外 I/O 且容易过期。NameNode 重启后可以通过 DataNode 的 BlockReport 重新构建 BlockMap，因此没有必要持久化。

13. HDFS 写流程

写流程题非常容易考，按这个顺序背：

1. Client 向 NameNode 请求创建文件。
2. NameNode 检查文件是否已存在、权限是否合法。
3. NameNode 为文件分配 block，并返回可写入的 DataNode 列表。
4. Client 按 block 切分数据，和多个 DataNode 建立 pipeline。
5. Client 把数据写给第一个 DataNode。
6. 第一个 DataNode 写本地，同时转发给第二个 DataNode。
7. 第二个 DataNode 写本地，同时转发给第三个 DataNode。
8. DataNode 逐级返回 ack 确认。
9. 一个 block 写完后继续写下一个 block。
10. 文件写完后 Client 通知 NameNode 关闭文件，NameNode 记录文件完成。

图可以画成：

```
Client
  |
  | 1. 请求创建文件
  v
NameNode
  |
  | 2. 返回 DN1, DN2, DN3
  v

Client -> DN1 -> DN2 -> DN3
        <-ack<-ack<-ack
```

pipeline 管道写入是重点。Client 不是分别给三个 DataNode 写三遍，而是写给 DN1，DN1 转发给 DN2，DN2 转发给 DN3。ack 反向返回。

考试写法：HDFS 写文件时，Client 先向 NameNode 请求创建文件，NameNode 检查权限和文件状态后返回用于保存副本的 DataNode 列表。Client 将文件切分为 block，并与这些 DataNode 建立 pipeline。数据先写入第一个 DataNode，再由它转发给后续 DataNode，各节点写入成功后反向返回 ack。所有 block 写完后，Client 关闭文件，NameNode 更新元数据。

14. 副本放置策略

HDFS 默认通常保存 **3 个副本**。副本放置策略要兼顾**可靠性、网络开销、读取性能**。

典型策略：

第一个副本：如果 Client 在集群内，尽量放在 Client 所在节点；如果 Client 不在集群内，就选一个合适的 DataNode。

第二个副本：放在不同机架的节点上。这样即使一个机架断电或交换机故障，数据仍然可用。

第三个副本：放在第二个副本同机架的另一个节点上。这样减少跨机架网络传输，同时保证一定容错。

可以记成：

副本1：本地节点
副本2：不同机架
副本3：副本2同机架的另一个节点

考试写法：HDFS 的副本放置策略通常会把第一个副本放在本地节点，第二个副本放在不同机架，第三个副本放在第二个副本所在机架的另一台机器上。这样既减少写入时的网络传输，又能在节点故障和机架故障时保证数据可靠性。

15. HDFS 容错机制

HDFS 的容错主要靠 **多副本、心跳、BlockReport、checksum、重新复制**。

多副本：每个 block 保存多个副本。某个节点坏了，还可以读其他副本。

心跳机制 Heartbeat：DataNode 定期向 NameNode 发送心跳，表示自己还活着。如果 NameNode 长时间收不到某个 DataNode 心跳，就认为它故障。

BlockReport：DataNode 定期向 NameNode 汇报自己保存了哪些 block。NameNode 用这个信息维护 BlockMap。

重新复制 Replication：如果某个 block 因 DataNode 故障导致副本数不足，NameNode 会安排其他 DataNode 复制新的副本，让副本数恢复到设定值。

checksum 校验：DataNode 上每个 block 有校验信息。读数据时如果发现 checksum 不对，说明数据损坏，可以改读其他副本，并触发修复。

NameNode 容错补充：早期 NameNode 是明显单点。后来的 Hadoop 支持高可用，一般用 Active NameNode 和 Standby NameNode，一主一备，配合共享日志和 ZooKeeper 进行故障切换。考试如果没有展开问，写“NameNode 早期是单点，后续可通过 HA 机制缓解”即可。

考试写法：HDFS 通过多副本机制实现数据容错。DataNode 定期向 NameNode 发送心跳和 BlockReport，NameNode 根据这些信息判断节点是否失效并维护 block 位置信息。当某个 DataNode 故障导致 block 副本数不足时，NameNode 会调度其他节点重新复制副本。读取时还可以通过 checksum 检测损坏数据，并改读其他副本。

16. 为什么 HDFS 不适合存储小文件？

这是高频题，要直接背答案。

原因一：NameNode 内存压力大。 每个文件、每个 block 都要在 NameNode 中维护元数据。小文件数量太多时，即使真实数据不大，也会产生大量元数据，占用 NameNode 内存。

原因二：访问开销大。 读小文件时，定位、调度、建立连接、磁盘寻道等固定开销相对很高，可能比真正读文件内容更耗时。

原因三：无法发挥大 block 顺序吞吐优势。 HDFS 的优势是大文件顺序读写。小文件太碎，批处理任务会产生大量小任务，调度成本变高。

注意： 小文件“不适合”不等于“不能存”。可以存，但效率差，NameNode 压力大。

考试写法： HDFS 不适合存储大量小文件。因为每个文件和 block 都需要 NameNode 维护元数据，小文件数量过多会占用大量 NameNode 内存；同时，小文件读取时寻道、定位和任务调度等固定开销相对数据量过大，无法发挥 HDFS 大 block 顺序读写的吞吐优势。

本章高频简答模板

1. 为什么需要 HDFS？ 单机文件系统的数据存储和计算能力受限于单台服务器。当数据规模增大时，会出现容量不足、I/O 瓶颈和机器故障导致数据不可访问的问题。HDFS 通过将大文件切分为 block，分布式存储在多个 DataNode 上，并使用多副本机制保证容错，适合大规模离线批处理场景。

2. HDFS 有什么特点？ HDFS 具有高容错性、适合批处理、适合大数据处理、可运行在普通服务器上的特点。它通过多副本自动恢复数据，通过暴露数据位置支持移动计算而非移动数据，适合 GB/TB/PB 级大文件的顺序读写和离线分析。

3. HDFS 有什么局限？ HDFS 不适合低延迟访问、小文件存取、并发写入和随机修改。因为 HDFS 为大文件顺序读写和高吞吐批处理优化，小文件会占用 NameNode 大量内存，随机修改也不符合 HDFS 一次写入、多次读取或追加写入的设计。

4. NameNode 和 DataNode 的区别？ NameNode 是主节点，负责管理文件系统命名空间、文件到 block 的映射以及 block 到 DataNode 的位置信息；DataNode 是从节点，负责在本地文件系统中存储真实 block，并处理数据读写请求。NameNode 管元数据，DataNode 管真实数据。

5. 为什么 HDFS 的 block 很大？ HDFS 面向大文件和顺序批处理，因此采用较大的 block。大 block 可以减少磁盘寻道次数，减少 NameNode 需要维护的 block 元数据数量，提高顺序读写吞吐量。但这也导致 HDFS 不适合大量小文件。

6. BlockMap 为什么不持久化？ BlockMap 记录的是 block 副本当前所在的 DataNode，这个信息会随着 DataNode 上下线、故障、复制、迁移而频繁变化。持久化会产生大量 I/O 且容易过期。NameNode 重启后可以通过 DataNode 的 BlockReport 重新构建 BlockMap，因此不需要持久化。

本章易混点

Block 大小 vs 文件大小： HDFS block 是文件切分单位，不是说小于 64MB 的文件一定占满 64MB 磁盘空间。

NameNode 不存真实数据： NameNode 存元数据，不存文件内容。真实数据在 DataNode 上。

DataNode 不知道完整文件： DataNode 只知道自己有哪些 block，不知道这些 block 属于哪个完整 HDFS 文件。

FsImage/EditLog vs BlockMap: **FsImage/EditLog** 持久化, 用于恢复命名空间; **BlockMap** 不持久化, 由 DataNode 汇报重建。

高吞吐 ≠ 低延迟: HDFS 适合一次处理大量数据, 不适合毫秒级频繁小请求。

本章看到题目怎么写

看到“**为什么需要分布式文件系统 / HDFS**”: 写单机瓶颈 + 大文件分布式存储 + 多副本容错。

看到“**HDFS架构**”: 画 NameNode / DataNode / Client, 写 NameNode 管元数据, DataNode 存 block, Client 先问 NN 再访问 DN。

看到“**为什么 block 大**”: 写减少寻道、提高吞吐、减少元数据、适合大文件批处理。

看到“**写流程**”: 写 Client 请求 NN → NN 返回 DN → pipeline 写 DN1→DN2→DN3 → ack → close。

看到“**为什么不适合小文件**”: 写 NameNode 内存压力 + 调度/寻道开销大 + 无法发挥顺序吞吐优势。